Experiment 1 — Pulse Code Modulation (PCM)

Visual understanding of PCM modulation and demodulation

1 Aim

Implement and study Pulse Code Modulation (PCM) modulation and demodulation.

2 Block Flow Diagram

From analog input to reconstructed output.



3 Working Principle / Steps

Main operations in PCM modulation and demodulation.

1 Sampling



- Measure the analog signal at regular intervals T_s.
- Convert continuous time signal to discrete-time samples.

2 Quantization



- Approximate each sample to the nearest amplitude level.
- Continuous amplitude values are mapped to a finite set of levels.

3 Binary Encoding



- Represent each quantized level using n-bit binary code.
- Generate a PCM sequence of bits.

4 Transmission



- Send the PCM bit stream over the communication channel.
- Suitable for storage and transmission with good noise immunity.

5 Decoding



- Convert received n-bit groups back to quantized levels.
- Reconstruct the sampled sequence at the receiver.

6 Reconstruction



- Apply a low-pass reconstruction filter to smooth the samples.
- Obtain an analog signal that approximates the original waveform.

4 Key Formulas

Important equations used in PCM.

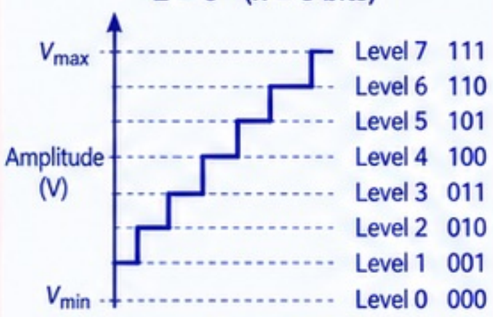
1	Sampling theorem	$f_s \geq 2f_m$	Sampling frequency must be at least twice the highest message frequency.
2	Number of levels	$L = 2^n$	L = number of quantization levels, n = bits per sample.
3	Quantization step size	$\Delta = \frac{V_{max} - V_{min}}{L}$	Δ = quantization interval or step size.
4	PCM bit rate	$R_b = n \times f_s$	R_b = number of bits transmitted per second.

5 Example: Quantization and Encoding

Illustrative example with 8 levels (3 bits/sample).

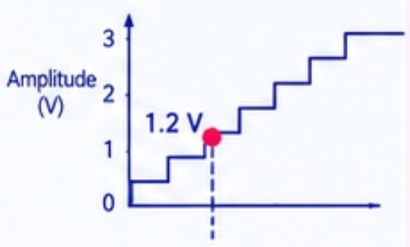
Quantization levels

$L = 8$ ($n = 3$ bits)



Encoding example

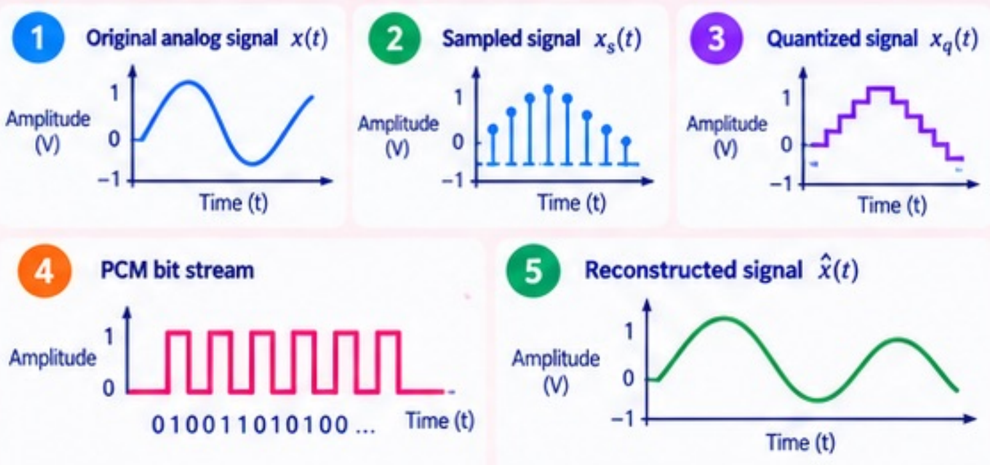
Sample value: 1.2 V



- Nearest quantized level: 1.0 V (Level 2)
- Binary code: 010

6 Expected Output / What to Observe

Waveforms at different stages of PCM system.



7 Viva Quick Points / Key Takeaways

1. Sampling converts a continuous-time signal into discrete-time samples.
2. Quantization maps each sample to a finite set of amplitude levels.
3. Binary encoding represents each quantized level with n bits.
4. PCM provides good noise immunity and is widely used in digital telecommunication systems.
5. The quality of the reconstructed signal improves with higher sampling rate and more quantization levels (more bits).

Experiment 6 — Direct Sequence Spread Spectrum (DSSS)

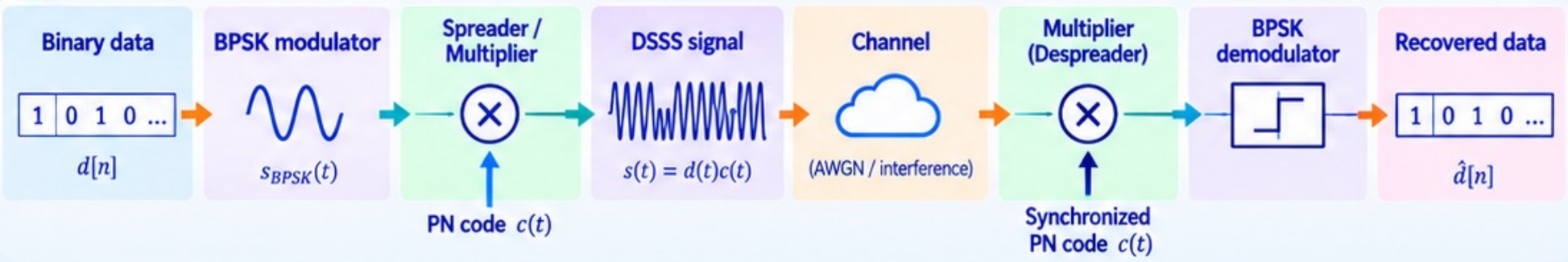
Visual understanding of DSSS spreading and despreading

1 Aim

Implement and study Direct Sequence Spread Spectrum (DSSS) in MATLAB/Python including spreading and despreading.

2 Block Flow Diagram

From binary input to recovered output using DSSS.



3 Working Principle / Steps

Main operations in DSSS modulation and demodulation.

1 Generate data bits

1 | 0 | ...

- Create a binary data sequence $d[n]$.
- Map bits to BPSK symbols $(+1 / -1)$.

2 Generate PN code

- Create a pseudo-random (PN) code $c[n]$ with high chip rate.
- Values are $+1 / -1$ (e.g. from an LFSR).

3 BPSK mapping

- Map $1 \rightarrow +1$ and $0 \rightarrow -1$.
- BPSK signal $s_{BPSK}(t) = d(t)$.

4 Spread signal

PN code $c(t)$

- Multiply the BPSK signal with PN chips bit by bit.
- $s(t) = d(t)c(t)$.

5 Transmit through channel

- Send DSSS signal over AWGN or interference channel.

6 Despread signal

Synchronized PN code $c(t)$

- Multiply the received signal with the same PN code.
- Spreading is removed.

7 Recover data

- Demodulate (BPSK) and make a decision.
- Recover the original binary sequence $\hat{d}[n]$.

4 Key Formulas

Important equations used in DSSS.

1	Chip rate vs data rate	$R_c > R_B$	Chip rate is higher than data rate ($R_c = NR_b$).
2	Spreading gain	$G_p = \frac{R_c}{R_b} = N$	G_p = processing gain (N = number of chips per bit).
3	DSSS signal (spreading)	$s(t) = d(t)c(t)$	Multiply data signal with PN code ($c(t) \in \{\pm 1\}$).
4	Despreading	$r(t) = s(t)c(t)$	Multiply again with the same PN code to remove spreading.

5 Example: Spreading with PN Code

Illustrative example for one bit using 7 chips ($N = 7$).

Data sequence (bits)

1 | 0 | 1

PN code (chips)

1 | 1 | 0 | 1 | 0 | 1 | 1

($N = 7$ chips/bit)

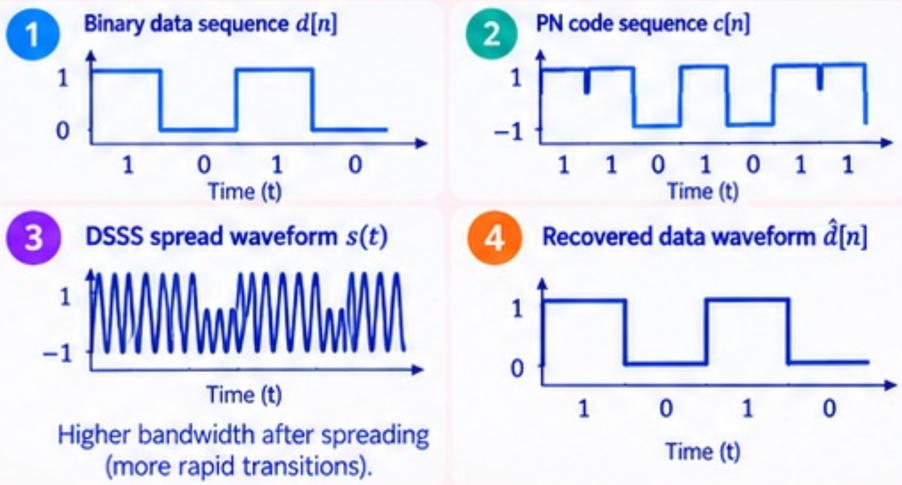
Spreading operation (one bit example)

Data bit 1 $(\rightarrow +1)$: 1 × PN code (1 1 0 1 0 1 1) → Spread chips (same as PN) (1 1 0 1 0 1 1)

Data bit 0 $(\rightarrow -1)$: 0 × PN code (1 1 0 1 0 1 1) → Spread chips (inverted) (−1 −1 0 −1 0 −1 −1)

6 Expected Output / What to Observe

Waveforms at different stages of DSSS system.



Higher bandwidth after spreading (more rapid transitions).

7 Viva Quick Points / Key Takeaways

1. DSSS spreads the signal spectrum over a wider bandwidth using a PN code.
2. It improves resistance to narrowband interference and jamming.
3. The same synchronized PN code must be used at the receiver for despreading.
4. Provides processing gain $G_p = R_c/R_b$, which improves signal reliability.
5. Bandwidth increases after spreading ($B \approx R_c$), compared to the original data bandwidth ($B \approx R_b$).

Experiment 7 — Frequency Hopping Spread Spectrum (FHSS)

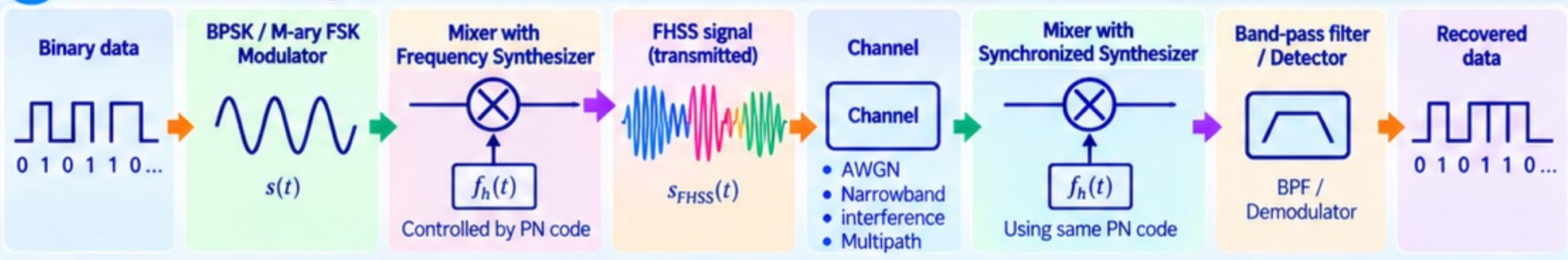
Visual understanding of frequency hopping and recovery

1 Aim

Implement Frequency Hopping Spread Spectrum (FHSS) modulation and demodulation in MATLAB/Python.

2 Block Flow Diagram

From binary data to recovered output using FHSS.



3 Working Principle / Steps

Main operations in FHSS modulation and demodulation.

1 Generate binary data

- Create a random bit sequence.
- Map bits to symbols (BPSK or M-ary FSK).

2 Generate hopping sequence

PN code

3 1 6 2 7...

- Use a pseudo-noise (PN) sequence to determine the hop frequencies.

3 Modulate the signal

- Modulate the binary data using BPSK or M-ary FSK.
- Generate baseband modulated signal $s(t)$.

4 Change carrier frequency

- Switch the carrier frequency according to the PN code.
- Each hop lasts for a fixed dwell time T_d.

5 Transmit hopped signal

- The signal is transmitted over the channel with changing carrier frequencies.

6 Synchronize at receiver

PN code (same seed)

- Generate the same hopping sequence at the receiver.
- Achieve time and frequency synchronization.

7 De-hop and detect data

- Mix with synchronized carrier, apply BPF and demodulate.
- Recover the original binary data.

4 Key Formulas / Concepts

Important equations and ideas in FHSS.

1	Hop frequency	$f_h = f_{min} + k\Delta f$	k is given by the PN sequence, $k = 0, 1, \ldots, N-1$
2	Dwell time	$T_d = \frac{1}{R_h}$	T_d = time spent at one frequency, R_h = hop rate (hops per second).
3	Processing gain	$G_p = \frac{B_{spread}}{B_{data}}$	Ratio of spread bandwidth to data bandwidth. Provides resistance to narrowband interference.
4	FHSS signal model	$s_{FHSS}(t) = s(t)\cos(2\pi f_h(t)t)$	Carrier frequency $f_h(t)$ changes according to the PN code.

5 Example: Hopping Sequence and Frequencies

Illustrative example with 7 possible hop frequencies.

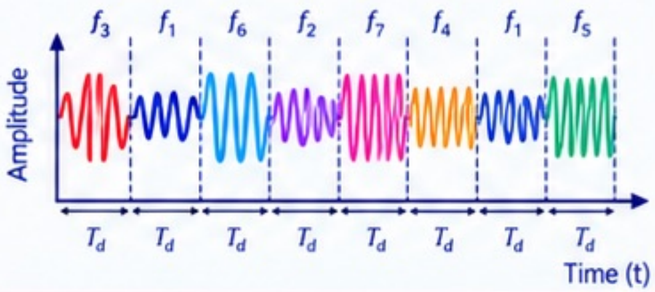
Available hop frequencies (N = 7)

$f_k = f_{min} + k\Delta f \quad (k = 0 \ldots 6)$

1	f_1	900 MHz
2	f_2	902 MHz
3	f_3	904 MHz
4	f_4	906 MHz
6	f_5	908 MHz
6	f_6	910 MHz
8	f_7	912 MHz

Example hop sequence for a message segment

Hop index (k)	3	1	6	2	7	4	1	5
Frequency	f_3	f_1	f_6	f_2	f_7	f_4	f_1	f_5



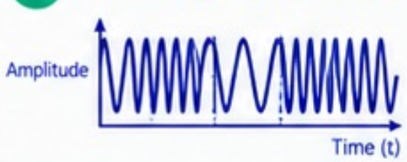
6 Expected Output / What to Observe

Waveforms at different stages of FHSS system.

1 Original bit sequence



2 Modulated signal (BPSK / FSK)



3 Spread signal with multiple frequencies



4 Received and recovered data



7 Viva Quick Points / Key Takeaways

1. Carrier frequency changes with time according to a PN hopping sequence.
2. FHSS spreads the signal over a wide bandwidth and provides resistance to narrowband interference and jamming.
3. Requires synchronization between transmitter and receiver (same PN code and timing).
4. Hop pattern (sequence of frequencies) is controlled by a pseudo-noise (PN) code.

Experiment 12 — Convolutional Codes

Visual understanding of convolutional encoding

1 Aim

Implement a convolutional encoder in MATLAB/Python.

2 Block Flow Diagram

From input bit stream to encoded output using shift registers and XOR logic.

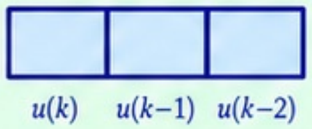
Input bit stream

1 0 1 1 …

$u(k)$
(one bit at a time)

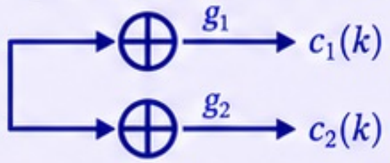
→

Shift registers (memory)



→

XOR combinational logic
(generator connections)



→

Encoded output pairs

c_1 c_2 c_1 c_2 …

($n = 2$ bits per input bit)

3 Working Principle / Steps

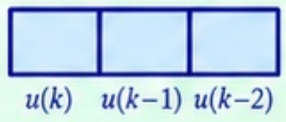
Main operations in convolutional encoding.

1 Input bit



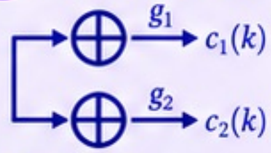
- Feed one input bit at a time from the message sequence $u(k)$.

2 Shift register



- Shift register stores the present bit and previous bits (memory).

3 Generator logic



- Output bits are produced using XOR connections specified by the generator polynomials.

4 Flush the encoder

0 0

- Append $(K-1)$ zeros at the end of the message to flush the encoder to the zero state.

5 Collect output

c_1 c_2 c_1 c_2 …

- Collect the encoded bit pairs to form the final encoded sequence.

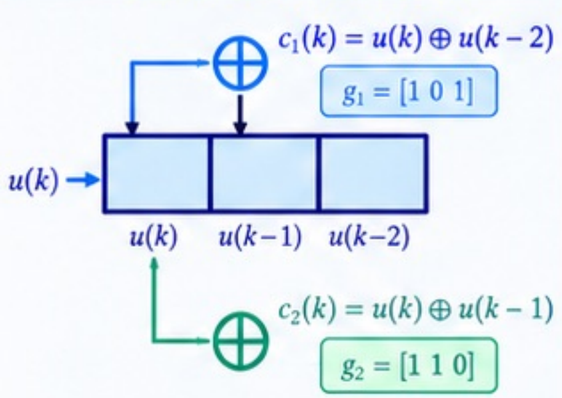
4 Key Formulas / Concepts

1	**Code rate**	$R = \frac{k}{n}$	For this experiment, (2,1,2) encoder: $k = 1$, $n = 2$ ⇒ $R = 1/2$.
2	**Constraint length**	$K = m + 1$	Memory order $m = 2$ ⇒ $K = 3$.
3	**Output equations**	$c_1(k),\ c_2(k)$	Each output bit depends on the current input bit and previous m bits in the shift register.
4	**General concept**		Convolutional code uses memory (previous bits) unlike block codes.

5 Example: Encoding a Message

Encode input message 1011 using a (2,1,2) convolutional encoder.

Encoder structure (example)



Step-by-step encoding

Time k	Input $u(k)$	Shift-register state $[u(k)\ u(k-1)\ u(k-2]$	Output bits $c_1(k)$	$c_2(k)$
0	1	1 0 0	1	1
1	0	0 1 0	1	0
2	1	1 0 1	0	0
3	1	1 1 0	0	1
4	0	0 1 1	0	1
5	0	0 0 1	1	1

Encoded output stream:

11 10 00 01 01 11

6 Expected Output / What to Observe

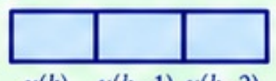
Verify the encoder operation and resulting code sequence.

1 Message bits

$u = 1\ 0\ 1\ 1$

(append 00)

2 Shift-register states



($K = 3$)

3 Output bit pairs

$c_1(k)$ $c_2(k)$

11 10 00 01 01 11

4 Final code vector

c = 11 10 00 01 01 11

(12 bits for 6 input bits)

7 Viva Quick Points / Key Takeaways

1. Convolutional codes add redundancy by producing multiple output bits for each input bit.
2. The redundancy helps in error detection and correction, improving reliability over noisy channels.
3. Uses memory (previous bits), unlike block codes which treat each block independently.
4. Viterbi decoding is commonly used at the receiver to decode convolutional codes efficiently.
5. Widely used in digital communication systems (e.g. satellite, wireless, and deep-space links).